RAASTA AI

Karachi Smart Traffic Management System

PRODUCT REQUIREMENTS PROPOSAL (PRP)

Simulate. Optimize. Prove.

A collaborative, simulation-first prototype for adaptive traffic signals, measurable performance comparisons, and preliminary infrastructure decision support.

Prepared for a 24+ student MERN development team
Hackathon delivery target: under 5 hours

Document overview

This PRP translates the RAASTA AI challenge into a buildable product scope, system architecture, simulation design, team structure, test plan and demonstration strategy. It is designed around a strict five-hour hackathon and a team of at least 24 students.

Contents

- 1. Executive summary and product vision
- 2. Problem statement, goals and non-goals
- 3. Similar systems and differentiation
- 4. Users, use cases and feature requirements
- 5. Recommended technology stack and architecture
- 6. Simulation model and adaptive signal algorithm
- 7. Metrics, experiments and infrastructure intelligence
- 8. Data strategy and assumptions
- 9. API contracts and repository structure
- 10. Team organization and collaboration workflow
- 11. Five-hour delivery plan
- 12. QA, acceptance criteria and risks
- 13. Demo script, presentation pitch and references

Document status: Proposed MVP specification. All simulation results are illustrative until generated by the implementation and validated against credible real-world data.

1. Executive summary

RAASTA AI is a Karachi-focused traffic simulation and decision-support web application. It allows users to compare conventional fixed-time traffic signals with an adaptive controller under the same simulated traffic demand. The product displays vehicle movement, queue lengths, waiting times, throughput, congestion hotspots and preliminary infrastructure options.

The central product concept is the **Karachi Traffic Decision Lab**: users can run controlled, repeatable experiments before considering operational changes or infrastructure investments. The project should demonstrate measurable outcomes from a functioning simulation rather than rely on static dashboards or fabricated AI claims.

Product promise: RAASTA AI does not merely display congestion. It lets a team test interventions, measure simulated effects, and explain which issues may be addressed by signal optimization versus those that warrant further engineering study.

2. Problem statement, goals and non-goals

Problem statement

Traffic congestion can be affected by uneven directional demand, signal timings that do not match current queues, turning conflicts, road bottlenecks and incidents. A dashboard alone cannot show whether a proposed intervention will help. A simulation can provide a controlled environment for comparing possible approaches.

Goals

- Build a working end-to-end web application in under five hours.

- Simulate at least four connected intersections with moving vehicles and signal phases.
- Implement fixed-time and adaptive signal-control modes.
- Compare both modes using identical demand, duration, road layout and random seed.
- Measure average waiting time, queue lengths, throughput and related metrics from simulation state.
- Flag persistent congestion hotspots and provide explainable preliminary intervention recommendations.
- Support a team workflow with clear ownership, a shared repository and repeatable tests.

Non-goals for the five-hour MVP

- Controlling real public traffic lights or connecting to live city infrastructure.
- Claiming citywide, real-time traffic coverage without verified live feeds.
- Training a deep reinforcement-learning model from scratch during the event.
- Making engineering approvals or claiming that a flyover/underpass is necessary based only on prototype metrics.
- Building a production-grade traffic digital twin of all Karachi roads.

3. Similar systems and differentiation

Existing tools and research show that traffic simulation and adaptive signal control are established fields. RAASTA AI should build on those ideas and differentiate itself through Karachi-focused scenarios, transparent comparisons, reproducibility and an explainable infrastructure decision workflow.

Reference	Relevance	How RAASTA AI can use it
SUMO (Simulation of Urban MObility)	Open-source microscopic traffic simulator; supports road networks, traffic lights and external control via TraCI.	Use as an optional realistic simulation engine if the team already has SUMO experience; otherwise use a small deterministic JavaScript engine for the MVP.
CityFlow	Open-source traffic simulation engine designed for large road networks and learning-based traffic control research.	Reference for multi-intersection modelling and programmatic signal control.
AI Traffic Management System (GitHub)	Example project combining traffic detection, simulation and dashboard concepts.	Study the architecture and feature organization; do not copy blindly or claim originality for existing components.
CityFlow_RL (GitHub)	Example of traffic-signal control experiments using SUMO/TraCI and Q-learning.	Use as a learning reference for controller evaluation, not as a requirement to train RL in five hours.
Karachi-related research and pilots	Prior research and reported smart-signal initiatives indicate local relevance and existing work in the field.	Position RAASTA AI as a prototype for reproducible scenario testing and decision support, not as the first adaptive traffic project.

Proposed differentiators

- **Karachi Scenario Lab:** named corridors/intersections with clearly labelled assumptions about demand and road geometry.
- **Peak-hour stress tests:** normal traffic, rush-hour surges, uneven directional demand and blocked approaches.
- **Explainable adaptive control:** show why a direction received priority, using queue, waiting-time and arrival indicators.
- **Infrastructure Decision Lab:** compare operational signal optimization with hypothetical road-layout or grade-separation scenarios.
- **Evidence mode:** store experiment settings, random seed, metrics and results so a judge can reproduce the comparison.

4. Users, use cases and feature requirements

Page / module	Priority	Requirements
Command Center	Must have	Show active scenario, vehicle count, average delay, throughput, congestion alerts and a map or network overview.
Traffic Simulator	Must have	Show four connected intersections, moving vehicles, signal states, Start/Pause/Reset, traffic intensity and Fixed/Adaptive mode.
Performance Lab	Must have	Compare fixed-time vs adaptive control under the same inputs; show waiting time, maximum queue, throughput and percentage changes.
Infrastructure Intelligence	Must have	Flag persistent hotspots and explain possible investigations: signal timing, lane/road redesign, turn restrictions or grade separation.
Emergency Corridor	Stretch	Simulate an emergency vehicle and a safe priority-control scenario after core features work.
Experiment History	Stretch	Save runs, settings and results; compare past runs and export a report.

Primary user journey

1. Select a Karachi-inspired scenario or a synthetic intersection network.
2. Choose traffic demand, simulation duration and control mode.
3. Run fixed-time control and record metrics.
4. Run adaptive control with the same demand, duration, road layout and random seed.
5. Compare measured results and inspect queues, signal phases and throughput.
6. Review hotspots and preliminary infrastructure options with their supporting indicators.
7. Save or export the experiment if time permits.

5. Recommended technology stack

Layer	Recommended choice	Responsibility
Frontend	React + Vite	Single-page web application and component architecture.
Styling	Bootstrap 5.3; optional Tailwind if already familiar	Responsive layout, controls, cards and visual consistency.
Charts	Recharts	Waiting time, throughput, queues and experiment comparisons.
Map	Leaflet + OpenStreetMap	Map context and location markers; road maps are not traffic-count data.
Backend	Node.js + Express	API, simulation coordination, metrics and recommendation logic.
Realtime	Socket.IO (optional for MVP)	Push live simulation state to connected clients.
Database	MongoDB + Mongoose	Persist scenarios, experiment configuration and results.
Simulation	JavaScript engine for fastest MVP OR Python + SUMO + TraCI	Vehicle movement, intersections, queues, signal phases and metrics.
Collaboration	GitHub + pull requests	Shared codebase, branch ownership and review.

Key implementation decision

Choose one simulation approach at the start. If a team member already knows SUMO/TraCI, use it. Otherwise implement a small deterministic JavaScript simulator and describe it honestly as a prototype. Avoid spending the majority of the event integrating multiple unfamiliar runtimes.

6. System architecture

Logical architecture

React + Vite frontend → REST API and optional Socket.IO → Node.js/Express orchestration → simulation engine → metrics and recommendation services → MongoDB persistence.

The frontend should render state received from the simulator, not invent its own signal states. A single source of truth should control vehicle positions, queues, current phases and displayed metrics.

Suggested main modules

- **Scenario service:** network layout, arrival rates, turn movements and simulation settings.
- **Simulation service:** ticks, vehicle movement, lane queues and intersection state.
- **Signal controller:** fixed-time and adaptive modes with safe phase transitions.

- **Metrics service:** delay, queue, throughput, stops and fairness calculations.
- **Experiment service:** paired runs, seeds, settings and comparison output.
- **Hotspot/recommendation service:** rule-based flags and explanation text.
- **Frontend visualization:** map/network, vehicles, signals, charts and controls.

7. Simulation model and adaptive signal algorithm

For a five-hour hackathon, a rule-based adaptive controller is recommended. It is easier to implement and explain than training a new neural network. It can still make demand-responsive decisions and be evaluated objectively.

Minimum simulation state

- **Intersection:** ID, connected approaches, current phase, phase start time and clearance state.
- **Approach/lane:** queue length, waiting-time accumulation, arrival rate and permitted movements.
- **Vehicle:** ID, position or lane index, arrival time, route and stopped/moving state.
- **Scenario:** road network, demand profile, duration, control mode and random seed.
- **Metrics:** total waiting, average delay, max queue, vehicles served, travel time and stopped count.

Illustrative demand-priority score

P■ = 0.6Q■ + 0.3W■ + 0.1A■

Q, W and A represent normalized queue length, waiting time and arrival rate for approach i. The coefficients are starting values for a prototype, not traffic-engineering standards. Normalize the variables so they are on comparable scales, and tune the weights through testing.

Controller cycle

1. Observe queue length, waiting time and arrivals on each approach.
2. Calculate a priority score for each eligible approach.
3. Respect minimum and maximum green durations and prevent conflicting green phases.
4. Apply amber and clearance intervals before changing conflicting movements.
5. Serve vehicles, update queues and accumulate metrics.
6. Repeat and record decisions so the controller can explain why it prioritized an approach.

Safety and fairness constraints

- Never switch directly between conflicting green phases.
- Enforce minimum green, maximum green, amber and all-red/clearance timing in the simulator.
- Prevent starvation by accounting for waiting time or maximum-wait constraints.
- Model compatible movements explicitly; a simplified four-way intersection should not assume every movement can run together.
- Describe the controller as a prototype, not as certified logic for real traffic infrastructure.

Fixed-time baseline

Use a preconfigured cycle with fixed green splits and clearance intervals. Both baseline and adaptive runs must use the same demand, network, duration and seed for a fair comparison. If possible, repeat across several seeds and report the mean and variability.

8. Metrics and experiment design

Metric	Definition / interpretation
Average waiting time	Total vehicle waiting time divided by the number of measured vehicles, using one consistent definition.
Average travel time	Mean elapsed time for vehicles that complete their routes; report incomplete vehicles separately.
Maximum queue length	Largest observed queue on any approach during the run.
Throughput	Vehicles that complete a defined movement or exit within the simulation period.
Stopped vehicles	Number of vehicles stationary at a given instant or cumulative stop events; clearly label which.
Fairness	Whether a direction or approach experiences excessive waiting compared with others.

Percentage improvement formula

Improvement (%) = ((Fixed metric – Adaptive metric) / Fixed metric) × 100

For metrics where lower is better (such as waiting time), a positive value indicates improvement. For throughput, reverse the comparison: ((Adaptive – Fixed) / Fixed) × 100. Handle zero denominators explicitly. Do not hardcode expected gains into the interface.

Required experiment scenarios

Scenario	Expected test purpose
Balanced demand	Check that adaptive control remains stable and does not create unnecessary switching.
Heavy north–south demand	Check whether a large queue gets priority without indefinitely starving other approaches.
Rush-hour surge	Measure whether the system detects and manages growing queues.
Blocked approach	Test hotspot detection, queue growth and explanatory operational recommendations.

9. Infrastructure Intelligence

The module should rank possible interventions for further investigation, not automatically prescribe construction. A flyover or underpass is a major engineering decision that requires traffic counts, origin-destination analysis, geometry, safety, access, environmental, cost and feasibility studies.

Intervention to investigate	Potential trigger in the prototype
Signal optimization	High delay and queues with evidence of poorly allocated green time or demand imbalance.
Lane / road redesign	Persistent queues associated with turning conflicts, lane bottlenecks or insufficient approach capacity.
Turn restrictions	A particular turning movement repeatedly blocks other traffic; access and safety impacts must be assessed.
Underpass / flyover	Severe bottleneck persists under tested operational changes; conduct a separate feasibility study.

Recommended output for every hotspot

- Location/scenario ID and observed simulation period.
- Evidence: queue duration, maximum queue, delay and throughput.
- Suggested intervention(s) and why they were flagged.
- Confidence or evidence-strength label based on available data, not an invented AI probability.
- Caveat: preliminary simulation output; professional traffic-engineering validation required.

Counterfactual comparison

Run the same demand under three explicit configurations: (A) baseline layout and fixed timings, (B) optimized signal timings, and (C) a hypothetical redesigned intersection. Compare the measured outcomes. Do not claim to simulate a real flyover unless its geometry, ramps, merges and traffic demand have actually been represented.

10. Data strategy and assumptions

Data type	Source / approach	Caution
Road geography	OpenStreetMap and Overpass Turbo; use road geometry and location labels where appropriate.	A road map is not a source of traffic volume or signal timings.
Traffic demand	Synthetic arrival rates for the MVP; document peak/off-peak assumptions.	Label all generated data as simulated.
Calibration / validation	Credible published studies, official counts or manually observed counts if available.	Record source, date, method and limitations.
Signal timing	Use documented assumptions or verified timings if available.	Do not present assumed timing plans as actual city signal settings.
Hotspots	Derived from persistent simulated queues and delay, with map locations as context.	Do not imply citywide live monitoring without a verified feed.

Useful resources

- OpenStreetMap — road geography and place context.
- Overpass Turbo — query OpenStreetMap features.
- SUMO documentation — simulation and traffic-control resources.
- SUMO tutorials — practical getting-started material.

- CityFlow — multi-agent traffic simulation research.
- CityFlow GitHub repository.
- AI Traffic Management System example repository.
- CityFlow_RL example repository.

Similar work in Karachi

Prior research and reported smart-signal pilots indicate that adaptive signal control is not a new concept in Karachi. The proposed differentiation is the reproducible, Karachi-focused comparison lab and explainable infrastructure workflow. Before a public presentation, verify the current status and exact details of any local pilot or research project from its original publication.

11. API contracts and repository structure

Suggested REST endpoints

Method and path	Purpose
GET /api/scenarios	List available scenario configurations.
POST /api/simulations/run	Start a simulation with scenario, mode, duration and seed.
POST /api/simulations/compare	Run or compare fixed-time and adaptive experiments with matched settings.
GET /api/simulations/:id	Retrieve simulation status and configuration.
GET /api/simulations/:id/metrics	Retrieve measured metrics and time series.
GET /api/hotspots	Retrieve hotspot candidates derived from simulation data.
POST /api/recommendations/evaluate	Evaluate hotspot evidence against documented rules.

Realtime event

If Socket.IO is used, define one stable event such as `simulation:update` for current tick, intersection states, vehicle positions, queue lengths and metrics. Keep the payload shape documented and consistent.

Suggested repository layout

```
raasta-ai/ ■■■ client/ ■ ■■■ src/ ■ ■■■ components/ ■ ■■■ pages/ ■ ■■■ simulation/ ■ ■■■ charts/ ■ ■■■ services/ ■■■ server/ ■ ■■■ src/ ■ ■■■ routes/ ■ ■■■ controllers/ ■ ■■■ services/ ■ ■■■ models/ ■ ■■■ simulation/ ■■■ shared/ ■ ■■■ types/ ■■■ docs/ ■■■ README.md ■■■ .gitignore
```

12. Team structure and ownership

For a 24-person team, use six groups with defined deliverables. For 25 or more students, add capacity to testing, UI polish or scenario development rather than creating duplicate implementations.

Team	Students	Ownership / deliverables
A — Frontend and UX	6	React app shell, sidebar, dashboard, simulator canvas, charts and responsive styling.
B — Simulation and adaptive logic	5	Vehicle movement, intersection state, signal phases, adaptive algorithm and fixed-time baseline.
C — Backend and database	4	Express APIs, simulation coordination, MongoDB schemas, experiment storage and Socket.IO.
D — Maps and infrastructure	3	Karachi scenario labels, hotspot rules, intervention logic and map integration.
E — QA and evaluation	3	Test scenarios, verify metric calculations, compare runs, integration checks and bug reports.
F — Integration and presentation	3	Repository structure, shared interfaces, deployment, documentation, pitch and live-demo coordination.
Total	24	One shared product and one shared codebase.

Collaboration rules

1. Create one shared GitHub repository and protect the main branch.
2. Assign one owner per feature/file area wherever possible.
3. Use short-lived feature branches, e.g. feature/adaptive-controller.
4. Merge through pull requests and review for compatibility.
5. Agree API request/response shapes before frontend and backend teams implement independently.
6. Commit small working changes frequently and keep the main branch demoable.
7. The integration lead coordinates dependencies but should not become the only person who understands the code.

13. Five-hour delivery plan

Time	Phase	Deliverable
0–30 min	Architecture and setup	Repository, React app, Express server, data contracts and working main branch.
30–100 min	Simulation core	Four intersections, arrivals, queues, fixed timing, adaptive timing and basic metrics.
100–165 min	Frontend integration	Dashboard, animated network, traffic controls, KPI cards and charts.
165–220 min	Comparison and intelligence	Paired experiments, hotspot rules and preliminary intervention recommendations.
220–260 min	Integration and QA	Test full journey, fix APIs, verify both modes and deploy if possible.
260–300 min	Feature freeze and demo	Rehearse demo, verify application, prepare backup recording and pitch.

Feature-freeze rule

At minute 260, stop adding features. Fix blockers, verify the primary user journey and rehearse. If time is lost, cut stretch features before cutting the working simulation or fair comparison.

14. QA, acceptance criteria and risks

Definition of done for the MVP

- Application starts from documented instructions.
- At least four connected intersections render and traffic moves through the network.
- Fixed-time and adaptive modes both run without crashing.
- The same scenario can be run with matched settings and seed in both modes.
- Metrics are calculated from simulation state, not hardcoded demo values.
- The comparison view shows actual output and handles zero/empty cases safely.
- Hotspot recommendations include the evidence and are labelled preliminary.
- Signal transitions respect the simulator's phase safety constraints.
- The team can explain limitations, assumptions and how to reproduce a run.

Risks and mitigations

Risk	Mitigation
Integration delays	Define contracts early; integrate a thin vertical slice by minute 100.
Too many features	Prioritize simulation + paired metrics; move emergency mode and export to stretch scope.
Misleading metrics	Define formulas once, test with small hand-calculated cases and never hardcode improvements.
Unrealistic traffic model	Label it as simulated, document assumptions and avoid claims of citywide accuracy.
Unsafe signal switching	Enforce minimum/maximum green and clearance transitions in controller logic.
Overconfident infrastructure claims	Recommend further study rather than construction; require professional validation.
Broken live demo	Keep a known-good main branch and prepare a short backup recording.

15. Demonstration and presentation strategy

Recommended three-minute demo

1. **Problem:** Show a Karachi-inspired intersection network under heavy demand.
2. **Baseline:** Run fixed-time control and observe queues form.
3. **Intervention:** Run adaptive control with the same demand and seed.
4. **Evidence:** Show actual differences in average delay, maximum queue and throughput.
5. **Insight:** Flag a persistent hotspot and compare signal optimization with a hypothetical layout change.
6. **Limitations:** Explain that simulated results require real-world data and professional traffic-engineering validation.

Suggested pitch

“RAASTA AI does not simply show Karachi's traffic congestion. It lets us test interventions before investing in them, measure their simulated effects, and explain which problems may be addressed by smarter signals and which may require further infrastructure studies.”

UI and presentation guidelines

- Use a consistent dark navy/teal palette, readable typography and restrained animation.
- Use clear labels for live simulation state, control mode, selected scenario and units.
- Show actual signal states from the engine; do not display static green/red lights disconnected from state.
- Make charts readable and use the same time period and definitions across comparisons.
- Label generated values as simulated and verified inputs with their source and date.
- Use a map for location context, but do not imply a live traffic feed unless one is connected and verified.

16. References and links

Technical documentation and example projects

- React documentation
- Express documentation
- Socket.IO documentation
- Leaflet documentation
- MongoDB documentation
- SUMO documentation
- SUMO tutorials
- CityFlow project
- CityFlow source repository
- AI Traffic Management System example
- CityFlow_RL example
- OpenStreetMap
- Overpass Turbo

Karachi context and related work

The earlier research and reporting discussed in the project-planning conversation included Karachi adaptive traffic control research, NED University Smart City Lab work, a reported PIDC smart-signal pilot, and research discussing 3 Talwar. Verify the original papers or current official announcements before quoting dates, implementation status, or specific performance claims in a public presentation.

17. Final recommendation

Build the Karachi Traffic Decision Lab as the core of RAASTA AI. Deliver four connected intersections, a fixed-time baseline, a demand-responsive adaptive controller, reproducible comparisons and explainable hotspot recommendations. Keep the implementation small enough to finish, and use the remaining time to improve reliability, visual polish and the quality of your demonstration.

Critical limitation: RAASTA AI is a decision-support prototype. It must not control real traffic infrastructure or be used to approve road construction without verified data, appropriate safety review and professional traffic-engineering validation.

End of PRP • RAASTA AI • Karachi Smart Traffic Management System